

Automated Printed Equation Recognition and LaTeX/MathML Conversion from Images



THESIS SUBMITTED TO
Symbiosis Institute of Geoinformatics

FOR PARTIAL FULFILLMENT OF THE M. Sc. DEGREE

By

Gurpreet Singh

(Batch 2023-2025 / PRN 23070243020)

Symbiosis Institute of Geoinformatics
Symbiosis International (Deemed University)
5th Floor, Atur Centre, Gokhale Cross Road,
Model Colony, Pune – 411016

CERTIFICATE

Certified that this thesis titled "**Automated Printed Equation Recognition and LaTeX/MathML Conversion from Images**" is a bona fide work done by Mr. Gurpreet Singh (23070243020) at Symbiosis Institute of Geoinformatics, under our supervision.

Supervisor, Internal

Name: Ms. Neeta Nandgude

Symbiosis Institute of Geoinformatics

UNDERTAKING

The thesis titled "**Automated Printed Equation Recognition and LaTeX/MathML Conversion from Images**" is a bona fide work by Mr. Gurpreet Singh. The University can use and reuse the dissertation work in the future.

Date: 11th July 2025

Name of the student: Gurpreet Singh

Signature

INDEX

S.NO	TITLE	PAGE NO
01	Introduction	1 - 3
02	Literature Review	4 - 7
03	Methodology	8 - 15
04	Results	16 - 21
05	Discussion	22 - 23
06	Conclusion	24
07	References	25 - 26
08	Appendix	27 - 30
09	Proforma 4	31
10	Plagiarism Report	32

ACKNOWLEDGEMENT

I want to express my heartfelt gratitude to my project guide, Ms. Neeta Nandgude, for her invaluable guidance, encouragement, and support throughout this research project. Her technical insights and constructive feedback were instrumental in shaping the direction and execution of my work.

I also wish to thank the Symbiosis Institute of Geoinformatics faculty and staff for providing an excellent academic environment and access to the resources necessary to complete this project successfully. I am grateful to my family and friends for their continuous encouragement and motivation during this journey.

Finally, I would like to acknowledge the authors and contributors of the datasets and open-source tools that formed the foundation for this research.

LIST OF TABLES

S.NO	TABLES	PAGE NO
01	Primary Libraries Used in the Project	8 - 9
02	Model Hyperparameters and Settings	12
03	Layer-wise Model Architecture Summary	14
04	Performance Comparison (Greedy vs Beam Search)	17

LIST OF FIGURES

S.NO	FIGURES	PAGE NO
01	Methodology Architecture	8
02	Preprocessed Equation Images	10
03	Distribution of Formula Sequence Length	11
04	Token Frequency Distribution	11
05	Greedy Search Decoding Samples	18
06	Beam Search Decoding Samples	18
07	Training Curve	19
08	UI Developed Using Streamlit	20

LIST OF ABBREVIATIONS

S.NO	ABBREVIATION	FULL FORM
01	OCR	Optical Character Recognition
02	UI	User Interface
03	CNN	Convolutional Neural Network
03	LWDSC-SA	Lightweight Depthwise Separable Convolution + Spatial Attention
04	BiLSTM	Bidirectional Long Short-Term Memory
05	RNN	Recurrent Neural Network
06	LSTM	Long Short-Term Memory
07	CRR	Character Recognition Rate
08	CER	Character Error Rate
09	WER	Word Error Rate
10	BLEU	Bilingual Evaluation Understudy
11	GPU	Graphics Processing Unit
12	API	Application Programming Interface
13	XML	Extensible Markup Language
14	LaTeX	Lamport TeX (Typesetting System for Scientific Documents)
15	MathML	Mathematical Markup Language
16	GT	Ground Truth
17	PT	PyTorch Model Checkpoint (.pt extension)
18	JSON	JavaScript Object Notation
19	ID	Identifier
20	CPU	Central Processing Unit
21	IO	Input/Output
22	B/W	Black and White (Image Format)
23	PNG	Portable Network Graphics
24	JPEG/JPG	Joint Photographic Experts Group
25	BMP	Bitmap Image File
26	TIFF	Tagged Image File Format
27	OTSU	Otsu's Method (Image Thresholding Technique)
28	PIL	Python Imaging Library
29	API	Application Programming Interface
30	ReLU	Rectified Linear Unit
31	ADAM	Adaptive Moment Estimation (Optimizer)
32	SOS	Start of Sequence
33	EOS	End of Sequence
34	ML	Machine Learning
35	DL	Deep Learning

PREFACE

This thesis documents the development and evaluation of an automated system for printed mathematical equation recognition and LaTeX/MathML conversion, undertaken as part of my six-month major project in the M.Sc. program in Data Science & Spatial Analytics at Symbiosis Institute of Geoinformatics, Pune.

The motivation behind this project stems from the increasing demand for accurate and efficient digitization of mathematical content, particularly in academic publishing, education, and document accessibility. Leveraging recent advances in deep learning and computer vision, this work explores the adaptation of novel neural architectures—specifically, the LWDSC-SA encoder, previously proposed for plant disease classification—to the domain of mathematical equation recognition.

Throughout this project, I have aimed to balance technical innovation with practical applicability, ensuring that the resulting system is accurate, efficient, user-friendly, and adaptable to real-world scenarios. The report outlines the comprehensive research process, including literature review, methodology, system design, implementation, evaluation, and findings.

I hope this work contributes meaningfully to ongoing research in automated document analysis and inspires further innovation in mathematical content recognition and digital accessibility.

INTRODUCTION

Mathematical expressions are the language of science, engineering, and quantitative analysis. Their precise representation, manipulation, and dissemination underpin research, education, and innovation in mathematics, physics, engineering, economics, and beyond. The proliferation of digital documents, remote learning platforms, and electronic publishing has elevated the demand for automated systems that can faithfully extract, interpret, and encode mathematical notation from diverse sources into structured, machine-readable formats such as LaTeX and MathML (**Chang et al., 2016; Genthial & Sauvestre, 2017; Xia et al., 2025**). Automating this process is crucial for academic publishing, digital archiving, and developing assistive technologies, advanced search engines, and computational tools that support mathematical reasoning and accessibility.

The problem of mathematical equation recognition and markup conversion presents challenges far beyond those encountered in traditional text recognition. Mathematical notation is inherently two-dimensional, with intricate spatial relationships, such as superscripts, subscripts, fractions, matrices, and multi-line expressions, that convey essential semantic meaning. Conventional Optical Character Recognition (OCR) methods, which are optimized for linear text, often fail to capture these complex spatial and structural dependencies, resulting in substantial loss of information or misinterpretation of content (**Öksüz et al., 2008; Chang et al., 2016; Orji et al., 2023**). Furthermore, mathematical symbols are highly context-dependent, and the same symbol may assume different roles depending on its position or surrounding elements (**Schechter et al., 2017; Kavyashree et al., 2018**).

To address these limitations, research over the last decade has increasingly focused on data-driven approaches leveraging deep learning. Early innovations involved the use of Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) to recognize isolated symbols or sequentially decode mathematical content (**Genthial & Sauvestre, 2017; More, 2018; Mishra & Kaur, 2018**). Encoder-decoder frameworks, particularly those enhanced with attention mechanisms, have enabled more robust end-to-end models capable of generating structured LaTeX or MathML markup directly from images of equations (**Jian Wang et al., 2019; Sundararaj et al., 2024; Garkal et al., 2021**). More recently, transformer-based architectures—adapted from natural language processing—have set new benchmarks by more effectively capturing both local and global spatial dependencies, leading to significant improvements in recognition accuracy and markup generation (**Patil et al., 2025; Xia et al., 2025; Gurgurov & Morshnev, 2024**).

In parallel, a concerted effort has been made to expand and diversify the datasets available for training and evaluating mathematical OCR systems. Benchmark datasets such as IM2LATEX-100K, CROHME, and MM-LaTeX, encompassing a wide array of printed, typeset, and handwritten equations, have been instrumental in driving progress and enabling objective comparisons between competing methodologies (**Genthial & Sauvestre, 2017; Hongyu Wang & Shan, 2020; Xia et al., 2025**). The availability of such datasets has supported the development of increasingly sophisticated models and facilitated the emergence of unified frameworks capable of handling printed, handwritten, and document-level mathematical content (**Singh & Karayev, 2021; Sundararaj et al., 2024**).

Despite these advancements, several open research challenges persist. Handwritten mathematical expressions, in particular, exhibit immense variability in character formation, symbol placement, and expression layout—factors that can confound even state-of-the-art

recognition systems (**Hongyu Wang & Shan, 2020; Garkal et al., 2021**). Real-world deployment scenarios, such as mobile phone capture, scanned archival documents, and low-quality images, introduce additional noise, distortions, and background clutter, further complicating the extraction and interpretation of mathematical notation (**Kavyashree et al., 2018; Francis & Sangeetha, 2025**). Additionally, issues related to symbol disambiguation, error propagation in structural parsing, and the seamless integration of recognition systems into user-friendly applications remain significant hurdles (**Patil et al., 2025; Orji et al., 2023**).

Recent research directions have begun to address these challenges by exploring advanced data augmentation strategies, domain adaptation, active learning, and multi-modal integration—combining visual, semantic, and contextual cues to enhance system robustness and generalizability (**Orji et al., 2023; Xia et al., 2025; Gurgurov & Morshnev, 2024**). Furthermore, attention is being paid to the accessibility and inclusivity of mathematical content, as seen in efforts to support Braille conversion and develop lightweight models deployable in resource-constrained environments (**Kavyashree et al., 2018; Batool et al., 2025**).

In light of this evolving landscape, the present work aims to contribute a robust, automated system for the recognition and markup conversion of printed mathematical equations. Leveraging state-of-the-art deep learning and transformer-based architectures, this project is designed to achieve high accuracy in LaTeX and MathML generation while maintaining computational efficiency suitable for real-world deployment. The system is developed and evaluated on contemporary benchmark datasets, emphasizing practical challenges such as symbol disambiguation, noise robustness, and output compatibility with standard academic and word-processing platforms.

By advancing the technical foundations and practical applications of mathematical equation recognition, this research seeks to bridge persistent gaps in automation, usability, and accessibility, contributing to academic knowledge and the broader scientific and educational communities that depend on accurate and efficient digital mathematics processing.

RESEARCH OBJECTIVES

This research aims to design, adapt, and evaluate an automated system for recognizing printed mathematical equations and converting them into LaTeX and MathML markup, supporting seamless integration with academic and word-processing platforms. The following objectives were systematically addressed and achieved in the course of this project:

1. Construct a Comprehensive Dataset Pipeline:

Assemble, preprocess, and augment a large-scale dataset of printed mathematical equations—principally utilizing the im2latex-100k dataset—to ensure broad coverage of equation types and symbol diversity. Implement robust data pipelines for image normalization, conversion to `.npy` format, vocabulary extraction, and LaTeX sequence tokenization.

2. Adapt and Integrate the LWDSC-SA Model for Equation Recognition:

For the first time in mathematical equation recognition, adapt and implement the Lightweight Depthwise Separable Convolution with Spatial Attention (LWDSC-SA) architecture originally proposed by **Batool et al. (2025)** for plant disease classification. Integrate this encoder with a BiLSTM sequence model and Luong-style attention decoder to create a computationally efficient yet accurate system for image-to-markup conversion.

3. Achieve High-Accuracy LaTeX/MathML Conversion:

Train and evaluate the adapted model for accurately generating LaTeX and MathML sequences from printed equation images. Assess system performance using BLEU score, Word Error Rate (WER), Character Error Rate (CER), Exact Match (EM), and edit distance, demonstrating competitive or state-of-the-art results across a range of equation complexities.

4. Optimize for Resource Efficiency and Scalability:

Ensure that both training and inference pipelines are computationally efficient, allowing the full development and deployment of the system on modest hardware such as an RTX 2060 GPU (6GB VRAM) without compromising recognition quality.

5. Develop an Interactive User Interface:

Build and deploy a user-friendly web application using Streamlit, enabling end users to upload images of printed equations and receive real-time LaTeX and MathML outputs, thereby validating the system's practical applicability.

6. Guarantee Output Compatibility and Integration:

Produce outputs in both LaTeX and MathML formats to facilitate seamless use in academic writing tools (e.g., Microsoft Word, Overleaf) and to support downstream applications, including automated grading and accessible document creation.

These objectives establish a strong foundation for advancing automated equation recognition and demonstrate the practical impact of adapting innovative deep learning architectures to new domains.

LITERATURE REVIEW

Mathematical equation recognition and markup conversion represent critical challenges at the intersection of computer vision, pattern recognition, and document analysis. With the proliferation of digital learning, scientific publishing, and accessible content creation, automating the extraction and semantic representation of mathematical expressions from printed and handwritten sources has become a central research goal. Over the past decade and a half, numerous approaches—ranging from classical recognition frameworks to advanced deep learning architectures—have been proposed to address this problem. This literature review surveys the chronological evolution of techniques, datasets, and models to convert mathematical equations into LaTeX or MathML, highlighting key advancements, persisting challenges, and the current state of the art.

Automating mathematical equation recognition and markup conversion has been a research focus for over a decade, with significant progress in handwritten and printed domains. Early efforts by **Öksüz et al. (2008)** introduced a video-based framework employing Finite State Machines (FSMs) to interpret pen-tip trajectories for handwritten equation recognition, achieving 92–93% character recognition rates. This system leveraged region and chain codes for online recognition and LaTeX generation, but faced limitations due to constrained input formats and challenges arising from ambiguous handwriting and pen-tip detection errors.

Transitioning to printed and typeset equations, **Chang et al. (2016)** utilized the Wikipedia Equation Dataset and InftyCDB-3B, combining recursive projection profile cutting for segmentation with CNN and SVM classifiers, supplemented by layout and grammar-based structural analysis. Their methodology yielded 92% accuracy in character segmentation and demonstrated that context-aware recognition and neural network-based structural analysis could enhance robustness in equation-to-LaTeX conversion.

Schechter et al. (2017) targeted handwritten mathematical expressions, implementing SVMs for segmentation, NNs and CNNs for character recognition, and heuristics for structure analysis. Their CNN model achieved 86% test accuracy, but reliance on heuristics for parsing led to brittleness, highlighting the need for integrated deep learning approaches to improve noise resilience and structural understanding.

Genthial and Sauvestre (2017) introduced an end-to-end encoder-decoder model, using a CNN and LSTM decoder with attention on the im2latex-100k dataset. Achieving a BLEU score of 78%, the model demonstrated the applicability of image captioning architectures for LaTeX sequence generation, with suggestions for improving encoders and extending the approach to handwritten formulas.

In a subsequent study, **More (2018)** developed a CNN-based approach for LaTeX code prediction from images of complex equations, achieving over 90% accuracy for sequences up to 35 characters. The work emphasized the limitations of pure CNNs in modeling spatial relationships, advocating for encoder-decoder architectures to capture structural dependencies more effectively.

Expanding the scope to accessibility, **Kavyashree et al. (2018)** proposed a system for converting printed mathematical equations into LaTeX and Braille. Employing adaptive thresholding, skew correction, and nearest-neighbor-based recognition, the system achieved

90% accuracy on ideal images but struggled with cluttered backgrounds and was limited to printed, not handwritten, inputs.

Mishra and Kaur (2018) leveraged the Image to LaTeX 100K dataset, integrating CNNs and LSTMs with peephole connections and a stochastic "hard" attention mechanism. Their model achieved a 75.87% accuracy and a BLEU score 75.08, illustrating the benefits of advanced recurrent and attention mechanisms. It also points to the need for adaptation to handwritten expressions and API deployment.

Jian Wang et al. (2019) advanced the encoder-decoder paradigm with a DenseNet-based encoder and a channel-spatial (C-S) joint attention mechanism, achieving a BLEU score of 88.25% and an exact match rate of 37.09% on the im2latex-100k dataset. The study noted the computational expense of DenseNet encoders and highlighted the need for greater efficiency in model design.

Efforts by **Hongyu Wang and Shan (2020)** addressed handwritten mathematical expressions using a "watch step-by-step" dense encoder, 2-D attention with coverage, and a GRU-based decoder. Evaluated on CROHME benchmarks, their framework achieved expression rates of 49.46% and 46.06%, demonstrating the effectiveness of transfer learning and regularization, though recognizing long expressions remained challenging.

Zelun Wang and Liu (2020) shifted focus to document-level conversion from PDF to LaTeX, combining projection profile cutting, CNNs, CRFs, and CNN-LSTM sequence translators. Their system achieved an edit distance rate of 81.1%, highlighting improvements in visual, semantic, and context modeling for mathematical OCR, while identifying persistent issues in the accurate conversion of complex expressions.

In the domain of full-page handwritten and printed text recognition, **Singh and Karayev (2021)** proposed an image-to-sequence model utilizing a ResNet encoder and Transformer decoder, achieving state-of-the-art recognition on the IAM dataset and surpassing commercial APIs on free-form handwritten tasks. The work underscored the importance of standardized benchmarks and scalable model architectures for long-sequence transcription.

Garkal et al. (2021) introduced a DenseNet-LSTM encoder-decoder architecture with variational dropout and attention for handwritten mathematical expression recognition on CROHME datasets, attaining 35.8% and 34.6% accuracy. Their application of variational dropout LSTMs addressed symbol ambiguity and expression parsing while highlighting ongoing structural analysis challenges.

Zelun Wang and Liu (2021) further refined encoder-decoder architectures for printed equations, employing a VGG-based CNN encoder with 2D positional encoding and bidirectional LSTM decoder, optimized via policy gradient reinforcement learning. Their system achieved a BLEU score of 90.28% and an exact match rate of 82.33% on IM2LATEX-100K, advancing spatial representation and sequence optimization.

A comprehensive survey by **Orji et al. (2023)** reviewed deep learning strategies, preprocessing, data augmentation with LaTeX constraints, and active learning in OCR for image-to-LaTeX conversion. The review emphasized the necessity for robust, adaptive preprocessing, domain-specific evaluation metrics, and noise-robust algorithms, advocating for further research into domain adaptation and active learning strategies.

Krishnan et al. (2023) presented a CNN-based approach for handwritten equation recognition and automatic solving using SymPy, achieving 97.56% accuracy on CROHME data and outperforming traditional models. Despite this, the study recognized the enduring challenges in symbol segmentation, complex notation, and efficiency in practical deployments.

Sundararaj et al. (2024) reported recent explorations into transformer-based architectures, which compared CNN-RNN, ResNet50-RNN, and Vision Transformer-based models on the im2latex datasets. Their transformer model outperformed baselines in loss, accuracy, and BLEU score, establishing the efficacy of multi-head attention mechanisms for equation-to-LaTeX tasks.

Gurgurov and Morshnev (2024) contributed an open-source image-to-LaTeX system employing a Swin Transformer encoder and GPT-2 decoder, further enhanced with Low-Rank Adaptation (LoRA) for efficient distributed training. Their model, trained on large-scale printed and handwritten data, achieved a Google BLEU score of 0.67, exemplifying the accessibility and scalability of transformer-based OCR pipelines.

In the context of real-world deployment, **Francis and Sangeetha (2025)** conducted a comparative analysis of widely used OCR tools on mobile phone images, finding EasyOCR to outperform others in precision and speed for both text and mathematical content, thereby addressing the need for robust, mobile-friendly OCR solutions.

Patil et al. (2025) validated the effectiveness of Vision Transformers with self-attention for mathematical document recognition and LaTeX conversion. Their transformer encoder-decoder, evaluated on the im2latex dataset, achieved a BLEU score of 92.00 and an exact match rate of 60.50. However, they continued to face challenges related to handwriting irregularities, dataset diversity, and usability.

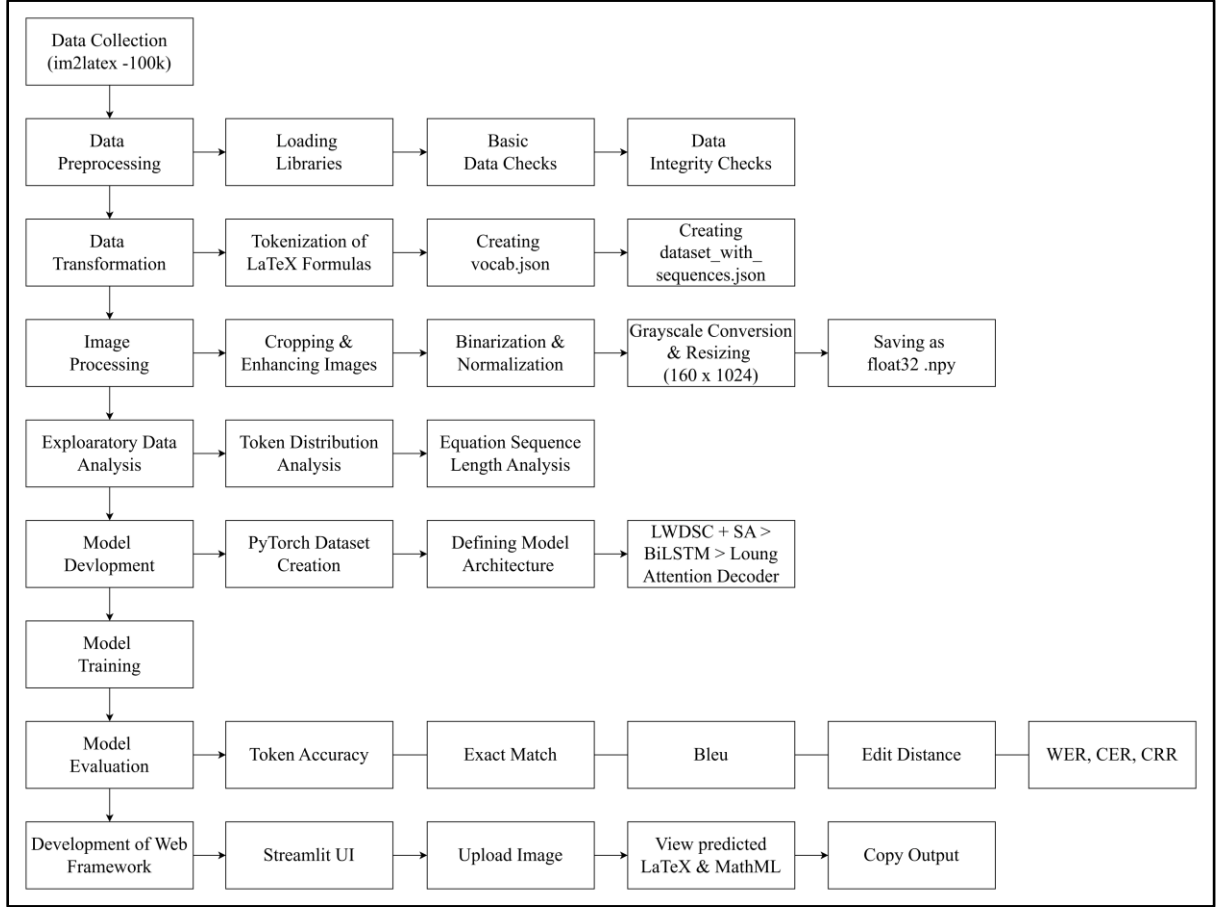
Xia et al. (2025) introduced LaTeXNet, which incorporates an image multiplexer and specialized Vision Transformer submodules to process tables, equations, and text equations. Trained on the extensive MM-LaTeX dataset, LaTeXNet outperformed state-of-the-art open-source and commercial models in all measured categories, establishing a new benchmark for unified document-to-LaTeX conversion and releasing an automated annotation tool and dataset for future research.

Finally, **Batool et al. (2025)** contributed insights from a related domain by proposing a lightweight deep learning model with depthwise separable convolutions and spatial attention for plant leaf disease classification. Their model achieved high efficiency and accuracy, demonstrating the applicability of attention-augmented lightweight models for resource-constrained tasks—an approach potentially transferable to equation recognition in real-world and embedded scenarios.

The literature demonstrates a clear progression from early rule-based and handcrafted feature extraction approaches toward increasingly sophisticated deep learning and transformer-based architectures. Each generational advance has yielded improvements in recognition accuracy, robustness to real-world input variation, and the ability to capture complex structural and spatial relationships inherent in mathematical notation. Despite these advancements, persistent challenges remain—including handling ambiguous handwritten symbols, achieving real-time efficiency, ensuring robustness to noise and layout variability, and improving the accessibility and usability of these systems for end-users. Future research will focus on scalable transformer

models, domain adaptation, user-friendly interfaces, and integrating multi-modal and contextual information to realize automated and universally accessible mathematical equation recognition and markup conversion.

METHODOLOGY



(Figure 1. Methodology Architecture)

This section presents a detailed description of the methodology developed for automated recognition and LaTeX/MathML conversion of printed mathematical equations from images, as implemented for this research. The process encompasses dataset acquisition and integrity checks, comprehensive preprocessing, advanced deep learning model design (adapting the LWDSC-SA encoder for the first time to equation recognition), model training, and rigorous validation. All code and computation were performed in Python 3.10.12 on a local desktop equipped with an AMD Ryzen 5 3600 CPU, 16GB RAM, and an NVIDIA RTX 2060 (6GB) GPU running Windows 11.

Libraries and Computing Environment

A range of open-source Python libraries was used to support the diverse requirements of this research, including deep learning, data processing, computer vision, and scientific evaluation. Table 1 summarizes the key libraries, their versions, and descriptions.

(Table 1. Primary Libraries Used in the Project)

S. No.	Library	Version	Description
01	torch	2.2.0	PyTorch is a core deep learning framework for model building, training, and inference.

S. No.	Library	Version	Description
02	NumPy	1.26.4	Efficient numerical computations, tensor operations, and image arrays.
03	OpenCV-python	4.9.0.80	Image reading, computer vision, preprocessing, and transformations.
04	pandas	2.2.2	Data manipulation, loading and saving CSV/JSON files, and tabular data management.
05	matplotlib	3.8.4	Visualization of data distributions, loss curves, and image previews.
06	tqdm	4.66.4	Real-time progress bars for iterative operations, enhancing workflow monitoring.
07	Pillow	10.3.0	Advanced image processing: enhancement, sharpening, and contrast.
08	nltk	3.8.1	A natural language toolkit was used to evaluate the BLEU score of predicted LaTeX sequences.
09	Levenshtein	0.25.1	Sequence alignment and edit distance computation for string similarity assessment.
10	torchinfo	1.8.0	Model summary and parameter inspection for PyTorch networks.
11	json, os, random, collections, copy, re	built-in	Essential utilities for file handling, string ops, randomization, and data structures.

Dataset Acquisition and Quality Assurance

The study used the im2latex-100k dataset (Zenodo.org), consisting of 93,887 pairs of equation images and corresponding LaTeX code. The dataset was split into training (75,243), validation (8,365), and test (10,279) partitions, each provided as a CSV file containing image and formula columns. Before any modelling, a comprehensive data validation procedure was undertaken:

- **Existence Check:** Every referenced image in each split was verified; no missing files were found.
- **Empty Image Check:** Using pixel-level statistics, each image was checked for empty, all-black, or all-white pixels; no such corrupt images were detected.
- **Duplicate Analysis:** Duplicates were checked within and across splits, confirming that all entries were unique.
- **CSV Consistency:** All splits were checked for consistent columns and correct headers.

This multi-stage validation ensured that only valid and unique image-formula pairs entered the modelling pipeline, laying a foundation for reproducibility and reliability.

Data Preprocessing

Tokenization and Vocabulary Construction

The formulas, written in LaTeX, were tokenized using space-based splitting, which preserves the structural elements critical to LaTeX syntax. Each tokenized sequence was augmented with special start-of-sequence <SOS> and end-of-sequence <EOS> tokens. Using the training split, a comprehensive vocabulary was constructed by counting the frequency of each token and assigning a unique index, ensuring that special tokens such as <PAD> (for padding), <UNK> (unknown tokens), and the sequence mentioned above markers were always included.

The resulting vocabulary (stored as vocab.json) contained 544 unique tokens, constructed solely from the training data to avoid information leakage. Allowing efficient integer encoding of formulas for neural sequence modelling. Encoding and decoding procedures were rigorously validated: each sequence was checked for round-trip fidelity (original formula $\leftrightarrow$ token IDs $\leftrightarrow$ decoded formula), and only 19 minor mismatches were found among nearly 94,000 samples, demonstrating robust preprocessing.

Sequence Encoding and Data Splitting

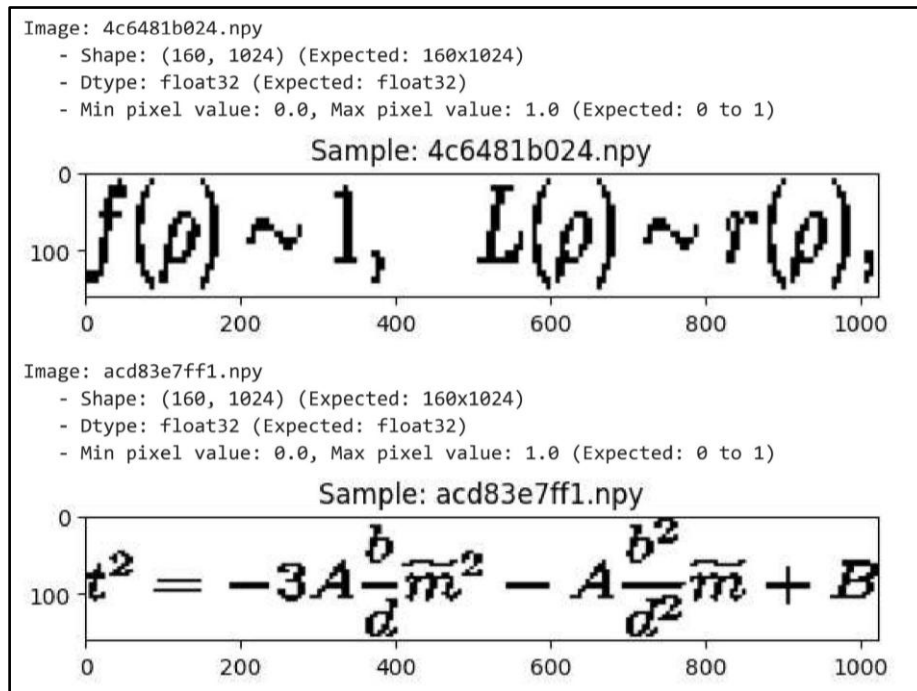
Each formula-image pair was encoded as a dictionary containing the processed image filename, the raw formula, the sequence of token IDs, and the split label. These records were compiled into dataset_with_sequences.json and subsequently partitioned into train_dataset.json, val_dataset.json, and test_dataset.json according to the provided splits. The entire data pipeline was validated for consistency, and random samples were visually and textually cross-checked to ensure correctness.

Given a formula string $f=t_1, t_2, \dots, t_n$, the tokenized representation is:

$$\text{Token IDs} = [\langle \text{SOS} \rangle, t_1, t_2, \dots, t_n, \langle \text{EOS} \rangle]$$

Image Processing and Normalization

Images were uniformly preprocessed to facilitate effective learning and computational efficiency. Each PNG image was loaded in grayscale and cropped to the content area by applying Otsu thresholding and bounding box extraction. After cropping, sharpness and contrast were enhanced using Pillow to amplify visual features essential for recognition. Images were binarized, resized to a fixed size of 160×1024 pixels (chosen to capture the elongated shape of most equations), and normalized to the $[0, 1]$ range. This standardization ensures that every image is presented to the model with consistent dimensions and intensity scaling. For optimal I/O speed, all images were saved as .numpy float32 arrays.

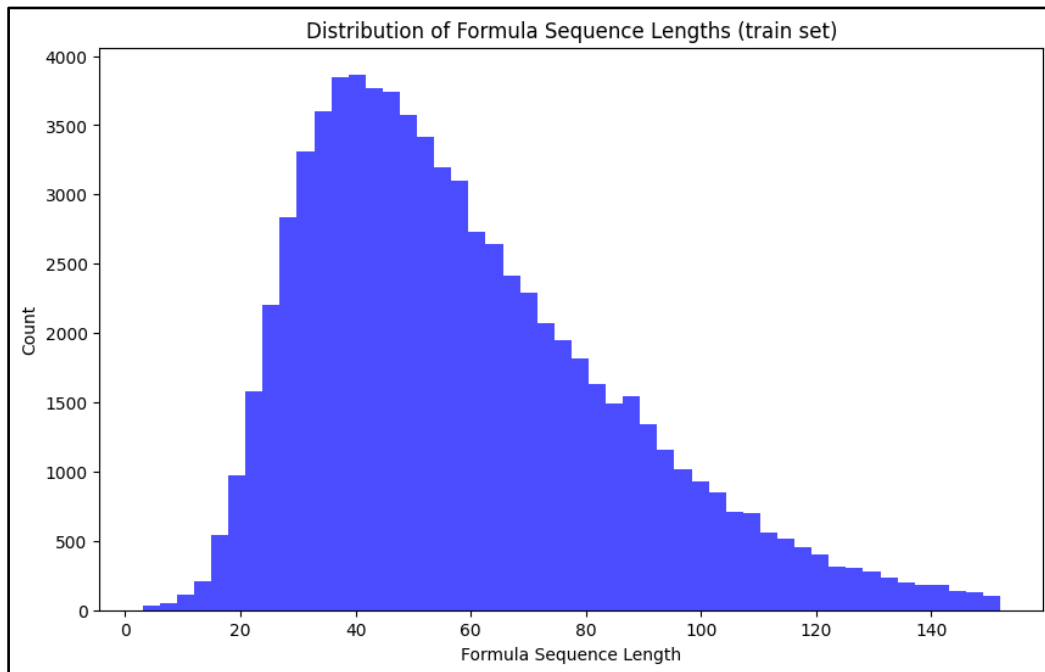


(Figure 2. Preprocessed Equation Images)

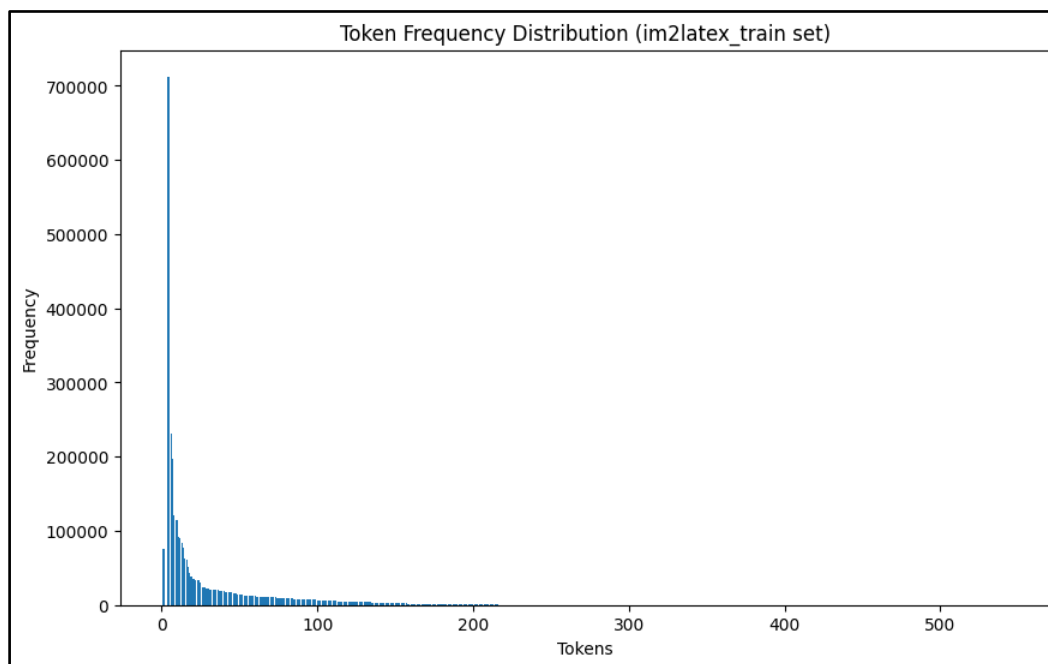
Exploratory Data Analysis

Formula Length and Token Distribution

Statistical analysis revealed that formula sequence lengths in the training set ranged from 3 to 152 tokens, with a mean of 58.8 and a median of 53. Token frequency analysis (via histogram and bar plots) showed a Zipfian distribution: a small number of tokens are widespread (e.g., structural characters $\{, \}, _, \wedge$, numerals), while many are rare, corresponding to specific math symbols or identifiers. This analysis informed vocabulary construction and guided architectural design, including the embedding size and model capacity.



(Figure 3. Distribution of Formula Sequence Length)



(Figure 4. Token Frequency Distribution)

PyTorch Dataset Construction

Custom PyTorch Dataset classes were implemented to streamline loading and batching. Each batch comprised a stack of image tensors [B, 1, 160, 1024] and a padded batch of token ID sequences, with actual lengths recorded for efficient processing. Padding was performed using the <PAD> token to the maximum sequence length in each batch, enabling seamless parallel computation on the GPU.

Model Hyperparameters

All model, training, and optimization hyperparameters are given in Table 2.

(Table 2: Model Hyperparameters and Settings)

Parameter	Value	Description
BATCH_SIZE	16	Training/validation batch size
HIDDEN_DIM	512	Decoder LSTM hidden size
EMB_DIM	384	Embedding dimension for tokens
ROW_BI_DIM	64	Row-wise BiLSTM hidden size
NUM_LAYERS	2	Number of layers in the sequence BiLSTM
DROPOUT	0.3	Dropout probability
LEARNING_RATE	1e-4	Initial Adam learning rate
LABEL_SMOOTH	0.2	Label smoothing epsilon
USE_FOCAL_LOSS	True	Focal loss enabled
GAMMA_FOCAL	2.0	Focal loss focusing parameter
NUM_EPOCHS	50	Max epochs for training
BEAM_WIDTH	5	Decoding beam width (in inference)
MAX_LEN	160	Maximum output sequence length
STRIDES	[2, 2, 1]	Strides in DSC encoder

Model Architecture

Encoder: Lightweight Depthwise Separable Convolution with Spatial Attention (LWDSC-SA)

The encoder is based on the LWDSC-SA design proposed by **Batool et al. (2025)** for plant disease classification, and it is used here for the first time in mathematical equation recognition. The encoder consists of three Depthwise Separable Convolution (DSC) blocks with increasing feature channels (32, 64, 128) and strides [2, 2, 1], each followed by batch normalization and ReLU activation. DSC reduces computational complexity, as the total parameter count for a DSC block is:

$$\text{Params}_{\text{DSC}} = D_k^2 \cdot M + M \cdot N$$

where D_k is the kernel size, M is the input channels, and N is the output channels. This is much lower than a standard convolution's parameterization.

After DSC, a Spatial Attention module is applied. The spatial attention computes a mask by concatenating the channel-wise average and max projections, passing them through a convolution and sigmoid function:

$$\text{SA}(X) = X \cdot \sigma(\text{Conv}_{k \times k}([\text{AvgPool}(X); \text{MaxPool}(X)]))$$

This attention mechanism ensures the encoder focuses on the most information-dense regions of mathematical notation.

Row-wise BiLSTM: Following the spatially attended feature map, a BiLSTM is applied to every row (height axis), treating each row as a sequence along the width. This yields, for each image of size $[B, 1, 160, 1024]$, a sequence output of shape $[B, W, H*2*\text{text}\{\text{ROW_BI_DIM}\}]$, where $\text{ROW_BI_DIM} = 64$. This module is essential for capturing the left-right structure typical of printed equations, such as fractions, roots, and multi-level expressions.

$$h_i = \text{BiLSTM}(\text{Row}_i)$$

Sequence Model: Bidirectional LSTM

The row-wise BiLSTM output is reshaped and passed through another BiLSTM, across the width dimension (with $\text{NUM_LAYERS} = 2$, $\text{HIDDEN_DIM} = 512$, $\text{dropout} = 0.3$). This module aggregates context from left-to-right and right-to-left directions, enabling the network to model dependencies across entire formula lines. The final output is a sequence suitable for the attention-based decoder.

Decoder: Luong Attention-Based Sequence Decoder

The decoder is a single-layer LSTM with an embedding layer ($\text{EMB_DIM}=384$) and employs Luong-style general (dot-product) attention. For each decoding step t , the attention mechanism computes alignment scores between the current decoder hidden state h_t and each encoder output h_i using a learnable weight matrix W :

$$\alpha_t = \text{softmax}(HW h_t)$$

where H is the matrix of all encoder outputs.

The attention context vector c_t is then calculated as a weighted sum of encoder outputs:

$$c_t = \sum_{i=1}^T \alpha_{t,i} h_i$$

At each time step, the context vector c_t is concatenated with the embedded input token and passed to the decoder LSTM. The LSTM output is projected to the vocabulary space (544 tokens) via a linear layer, yielding the predicted token logits. Teacher forcing is used during training, where the true previous token is provided as the following input.

Model Summary

A complete model summary is presented in Table 3, generated with the torchinfo library, illustrating every module's shape and parameter count.

(Table 3: Layer-wise Model Architecture Summary)

Layer (type:depth-idx)	Input Shape	Output Shape	Parameters
FullIm2LatexModel	[2, 1, 160, 1024]	[2, 544]	--
└─ LWDSCSA_Encoder: 1-1	[2, 1, 160, 1024]	[2, 256, 5120]	--
└─ Sequential: 2-1	[2, 1, 160, 1024]	[2, 128, 40, 256]	--
└─ DepthwiseSeparableConv: 3-1	[2, 1, 160, 1024]	[2, 32, 80, 512]	105
└─ DepthwiseSeparableConv: 3-2	[2, 32, 80, 512]	[2, 64, 40, 256]	2,464
└─ DepthwiseSeparableConv: 3-3	[2, 64, 40, 256]	[2, 128, 40, 256]	9,024
└─ SpatialAttention: 2-2	[2, 128, 40, 256]	[2, 128, 40, 256]	--
└─ Conv2d: 3-4	[2, 2, 40, 256]	[2, 1, 40, 256]	98
└─ Sigmoid: 3-5	[2, 1, 40, 256]	[2, 1, 40, 256]	--
└─ LSTM: 2-3	[80, 256, 128]	[80, 256, 128]	99,328
└─ SequenceModel: 1-2	[2, 256, 5120]	[2, 256, 1024]	--
└─ LSTM: 2-4	[2, 256, 5120]	[2, 256, 1024]	29,376,512
└─ LuongDecoder: 1-3	[2]	[2, 544]	--
└─ Embedding: 2-5	[2]	[2, 384]	208,896
└─ Linear: 2-6	[2, 512]	[2, 1024]	525,312
└─ LuongAttention: 2-7	[2, 1024]	[2, 1024]	--
└─ LSTM: 2-8	[2, 1, 1408]	[2, 1, 512]	3,936,256
└─ Linear: 2-9	[2, 512]	[2, 544]	279,072
Total Parameters			34,437,067
Trainable Parameters			34,437,067
Non-trainable Parameters			0
Total Mult-Adds (G)			17.32
Input Size			1.31 MB
Forward/Backwards Pass Size			146.61 MB
Parameter Size			137.75 MB
Estimated Total Size			285.67 MB

All parameter counts correspond to batch size 2 for summary purposes. Actual training batch size: 16.

Loss Function and Optimization

The model was trained with a custom loss function combining label smoothing and focal loss. Label smoothing with a factor $\epsilon = 0.2$ reduces overconfidence by distributing some probability mass to non-target classes. For vocabulary size K , smoothing factor ϵ :

$$\tilde{y}_k = \begin{cases} 1 - \epsilon & \text{if } k = y \\ \frac{\epsilon}{K - 1} & \text{if } k \neq y \end{cases}$$

Focal loss, with $\gamma = 2.0$, down-weights easy examples and focuses learning on more complex, misclassified examples:

$$\mathcal{L}_{\text{focal}}(p, y) = - \sum_{k=1}^K \tilde{y}_k (1 - p_k)^{\gamma} \log(p_k)$$

where p_k is the model's predicted probability for class k .

The Adam optimizer was used with a starting learning rate of 1×10^{-4} , and learning rate scheduling was managed by ReduceLROnPlateau (factor: 0.2, patience: 7). Early stopping was enforced if validation loss failed to improve for seven epochs.

Model Training Procedure

The model was trained for up to 50 epochs, each with a complete pass over the training data and subsequent validation. During each batch, images were fed through the encoder, followed by the sequence model and decoder, using teacher forcing to guide the sequence prediction. The loss was computed only on non-padding tokens. Each training epoch required approximately 47 minutes, and validation epochs averaged 3 minutes, reflecting the computational demands of sequence modeling on large-scale image-text data. After each epoch, validation was performed identically, but without parameter updates.

All metrics and losses were logged, and the best model weights (measured by lowest validation loss) were saved. Training progress was monitored using loss curves (see Results section), and detailed debug outputs of tensor shapes ensured correct data flow.

Data Integrity and Quality Assurance

At each stage, from initial data checks to post-processing, random sampling and round-trip encoding-decoding validated the consistency of both images and LaTeX formulas. All errors were logged for traceability, and final data splits were verified by random visual and token-level inspection. This rigour ensured the model and evaluation were grounded on reliable, reproducible data.

RESULTS

This section comprehensively evaluates the proposed printed/digital equation recognition system. The assessment encompasses both the quantitative performance of the deep learning model and the qualitative usability of the deployed user interface. The model was evaluated using greedy and beam search decoding strategies, while the Streamlit-based UI was tested for functional robustness and user experience. Together, these components demonstrate the practical viability of the system in academic and publishing applications.

Evaluation Metrics

To ensure a thorough analysis of model performance, the following evaluation metrics were employed:

1. Token Accuracy (%):

Measures the proportion of correctly predicted tokens from the total ground truth tokens.

$$\text{Token Accuracy} = \frac{\text{Correct Tokens}}{\text{Total Tokens}} \times 100$$

2. Exact Match (%):

A stricter metric, this calculates how many predicted LaTeX sequences exactly match the ground truth without any deviation.

$$\text{Exact Match} = \frac{\text{Exact Predictions}}{\text{Total Samples}} \times 100$$

3. BLEU Score (Bilingual Evaluation Understudy):

Evaluates the n-gram overlap between predicted and reference LaTeX sequences. BLEU-4 is used in this work.

$$\text{BLEU} - 4 = BP \cdot \exp \left(\sum_{n=1}^4 w_n \cdot \log p_n \right)$$

Where:

pn = modified n-gram precision, wn = weights (usually 0.25 each), BP = brevity penalty

4. Word Error Rate (WER) (%):

The word-level differences (insertions, deletions, substitutions) between prediction and ground truth are calculated.

$$\text{WER} = \frac{S + D + I}{N} \times 100$$

Where:

S = substitutions, D = deletions, I = insertions, N = total words

5. Edit Distance and Normalized Edit Distance:

Measures the minimum character-level transformations required to convert the predicted sequence into the ground truth.

$$\text{Normalized Edit Distance} = \frac{\text{Edit Distance}}{\text{Length of Ground Truth}} \times 100$$

6. Character Error Rate (CER) and Character Recognition Rate (CRR):

CER reflects the percentage of incorrectly predicted characters. CRR is its complement:

$$\text{CRR} = 100\% - \text{CER}$$

Quantitative Evaluation

The model was evaluated on the test dataset using greedy and beam search decoding. Table 4 presents a detailed comparison of the results.

(Table 4: Performance Comparison (Greedy vs Beam Search))

Metric	Greedy Decoding	Beam Search Decoding
Token Accuracy (%)	5.33	5.11
Exact Match (%)	33.43	30.90
BLEU Score	86.00	84.99
Word Error Rate (WER) (%)	9.96	10.03
Edit Distance (Mean)	21.49	23.15
Edit Distance (Median)	5.00	8.00
Normalized Edit Distance (Mean) (%)	8.82	9.21
Normalized Edit Distance (Median) (%)	4.06	4.63
Character Error Rate (CER) (Mean) (%)	10.15	10.36
Character Error Rate (CER) (Median) (%)	4.26	4.85
Character Recognition Rate (CRR) (Mean) (%)	89.85	89.64
Character Recognition Rate (CRR) (Median) (%)	95.74	95.15

Greedy decoding slightly outperformed beam search across most metrics, which is somewhat unconventional. Typically, beam search improves accuracy due to more exhaustive search space exploration; however, in this case, it may have led to local optima or incorrect prioritization of token sequences. The CRR of approximately 90% and a BLEU score above 85 reflect strong global syntactic accuracy, even if exact token-level alignment remains challenging.

Qualitative Analysis

Sample outputs from both decoding strategies are presented below to illustrate model behaviour:

Greedy Decoding:

```
Sample 1 [Greedy]
GT   : \alpha _ { 1 } ^ { r } \gamma _ { 1 } + \dots + \alpha _ { N } ^ { r } \gamma _ { N } =
0 \quad ( r = 1 , . . . , R ) \; ,
PRED : \alpha _ { 1 } ^ { r } \gamma _ { 1 } + \ldots + \alpha _ { N } ^ { r } \gamma _ { N }
= 0 \quad ( r = 1 , . . . , R ) \; ,
Edit Distance: 1 | Norm Edit Distance: 0.82%
=====
Sample 2 [Greedy]
GT   : \eta = - \frac { 1 } { 2 } \operatorname { l n } \left( \frac { \operatorname { c o s h }
} { \left( \sqrt { 2 } b _ { \infty } \sqrt { 1 + \alpha ^ { 2 } } \right) } \; y - \operatorname { a r c s i n }
h \right) \; \alpha \right) \} \{ \sqrt { 1 + \alpha ^ { 2 } } \} \right)
PRED : \eta = - \frac { 1 } { 2 } \operatorname { l n } \left( \frac { \operatorname { c o s h }
} { \left( \sqrt { 2 } b _ { \infty } \sqrt { 1 + \alpha ^ { 2 } } \right) } \; y - \operatorname { a r c s
i n h } \alpha \right) \} \{ \sqrt { 1 + \alpha ^ { 2 } } \} \right)
Edit Distance: 16 | Norm Edit Distance: 6.50%
=====
Sample 3 [Greedy]
GT   : P _ { ( 2 ) } ^ { - } = \int \beta d \beta d ^ { 9 } p d ^ { 8 } \lambda \Phi ( - p , -
\lambda ) \left( - \frac { p ^ { I } p ^ { I } } { 2 \beta } \right) \Phi ( p , \lambda ) \; ,
PRED : P _ { ( 2 ) } ^ { - } = \int \beta d \beta d ^ { 9 } p d ^ { 6 } \lambda \Phi ( - p , -
\lambda ) \left( - \frac { p ^ { I } p ^ { \prime } } { 2 \beta } \right) \Phi ( p , \lambda )
\; ,
Edit Distance: 7 | Norm Edit Distance: 3.85%
```

(Figure 5. Greedy Search Decoding Samples)

Beam Search Decoding:

```
Sample 1 [Beam]
GT   : \alpha _ { 1 } ^ { r } \gamma _ { 1 } + \dots + \alpha _ { N } ^ { r } \gamma _ { N } =
0 \quad ( r = 1 , . . . , R ) \; ,
PRED : \alpha _ { 1 } ^ { r } \gamma _ { 1 } + \ldots + \alpha _ { N } ^ { r } \gamma _ { N }
= 0 \quad ( r = 1 , . . . , R ) \; ,
Edit Distance: 1 | Norm Edit Distance: 0.82%
=====
Sample 2 [Beam]
GT   : \eta = - \frac { 1 } { 2 } \operatorname { l n } \left( \frac { \operatorname { c o s h }
} { \left( \sqrt { 2 } b _ { \infty } \sqrt { 1 + \alpha ^ { 2 } } \right) } \; y - \operatorname { a r c s i n }
h \right) \; \alpha \right) \} \{ \sqrt { 1 + \alpha ^ { 2 } } \} \right)
PRED : \eta = - \frac { 1 } { 2 } \operatorname { l n } \left( \frac { \operatorname { c o s h }
} { \left( \sqrt { 2 } b _ { \infty } \sqrt { 1 + \alpha ^ { 2 } } \right) } \; y - \operatorname { a r c
s i n h } \alpha \right) \} \{ \sqrt { 1 + \alpha ^ { 2 } } \} \right)
Edit Distance: 12 | Norm Edit Distance: 4.88%
=====
Sample 3 [Beam]
GT   : P _ { ( 2 ) } ^ { - } = \int \beta d \beta d ^ { 9 } p d ^ { 8 } \lambda \Phi ( - p , -
\lambda ) \left( - \frac { p ^ { I } p ^ { I } } { 2 \beta } \right) \Phi ( p , \lambda ) \; ,
PRED : P _ { ( 2 ) } ^ { - } = \int \beta d \beta d ^ { 9 } p d ^ { 6 } \lambda \Phi ( - p , -
\lambda ) \left( - \frac { p ^ { I } p ^ { \prime } } { 2 \beta } \right) \Phi ( p , \lambda )
\; ,
Edit Distance: 7 | Norm Edit Distance: 3.85%
```

(Figure 6. Beam Search Decoding Samples)

While most outputs are syntactically correct, minor semantic mismatches like symbol swaps or power index errors occur. These do not drastically affect the BLEU score but reduce exact match and token accuracy.

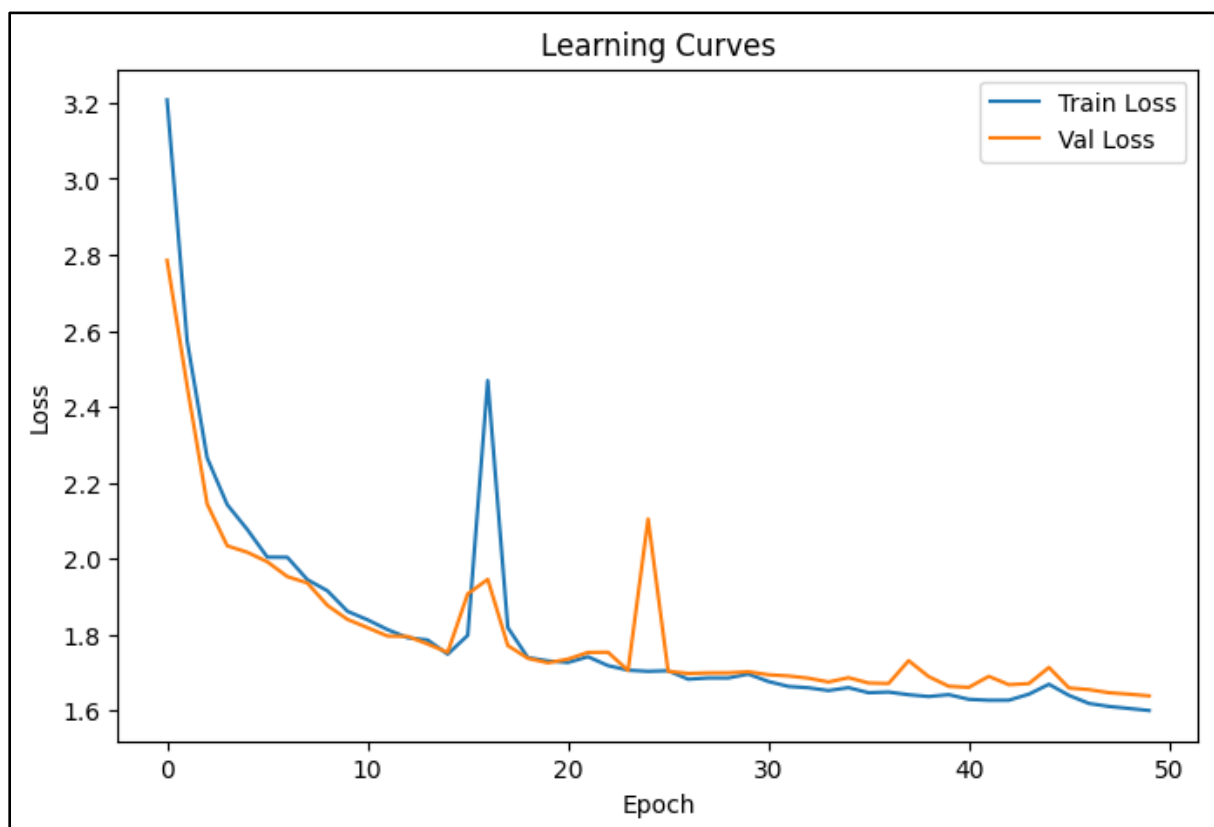
Error Analysis

Several trends emerged from the evaluation:

- Token accuracy is significantly low (~5%), implying challenges in strict token-wise alignment, likely due to expression length and formatting inconsistencies.
- BLEU and CRR scores remain high, indicating the model effectively captures structural patterns, even if a few tokens are incorrect.
- Median edit distance values are low, showing that predictions are only a few characters off in most samples.

These suggest the model may benefit from sequence-aware loss functions or more context-preserving decoding strategies.

Learning Dynamics



(Figure 7. Training Curve)

The training curve exhibits three distinct phases:

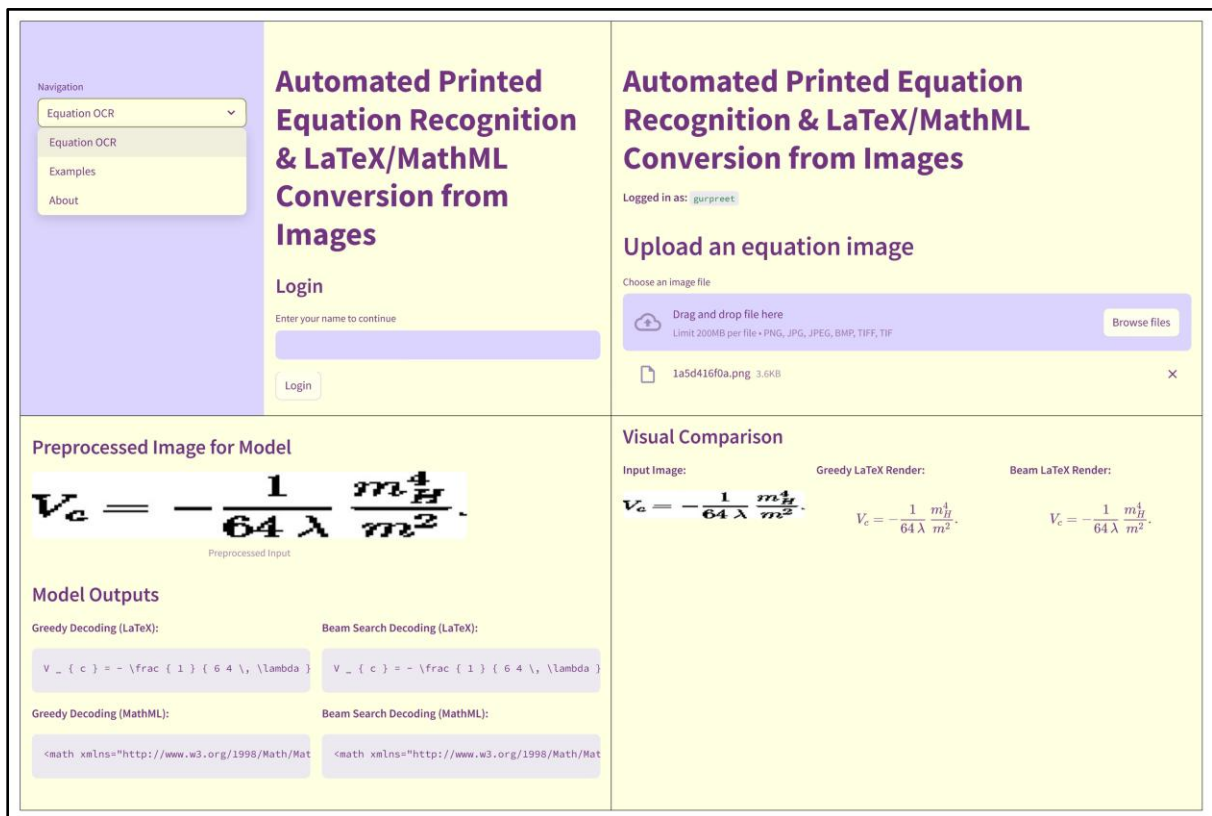
- **Initial Convergence (Epochs 0–10):** Rapid reduction in training and validation losses due to effective parameter learning.

- **Mid-training Instability (Epochs 15–25):** Temporary spikes likely caused by optimizer adjustments or data loader shuffling.
- **Final Stabilization (Epochs 25–50):** Losses stabilize around 1.65–1.70, with minimal overfitting observed.

The small generalization gap between training and validation losses indicates effective regularization.

User Interface (Streamlit)

A dedicated web-based user interface (UI) was developed using Streamlit, a Python framework ideal for rapidly building interactive ML applications. This UI enables end users to upload printed or digital mathematical equation images and receive predicted LaTeX and MathML outputs from a locally deployed deep learning model. It provides a structured, intuitive layout for image upload, inference visualization, and comparison of decoding strategies.



(Figure 8. UI Developed Using Streamlit)

UI Features and Workflow:

- **Navigation:** Three main sections were created: "Equation OCR" (core interface), "Examples" (placeholder for demos), and "About" (project details).
- **Login System:** A session-state-based name input system tracks user identity across pages.
- **Image Upload:** Users can upload images in various formats (.png, .jpg, .bmp, .tiff). Uploaded photos are preprocessed via grayscale conversion, binarization, cropping, and resizing (160x1024) before being passed to the model.

- **Model Architecture Loading:**

The model architecture consists of three main components, all loaded dynamically upon UI startup using `@st.cache_resource` for performance:

- **LWDSC-SA Encoder:** Lightweight CNN using depthwise separable convolutions and spatial attention
- **BiLSTM Sequence Model:** Captures horizontal spatial dependencies.
- **Luong Attention Decoder:** Decodes features into LaTeX tokens using attention.

The UI loads trained model weights from a checkpoint (`full_checkpoint.pt`) and corresponding vocabulary from `vocab.json`.

This ensures that the heavy lifting (model initialization and weight loading) is done only once per session unless explicitly invalidated.

- **Inference:** Both greedy and beam decoding strategies are run simultaneously. Predictions are converted to:
 - Plain LaTeX (via inverse vocabulary lookup)
 - MathML (via `latex2mathml` library)
 - Visual Render (via `st.latex()`)
- **Output Display:** A 3-column layout presents the preprocessed input, the LaTeX result from greedy decoding, and the result from beam search. Raw MathML is also shown for copy-paste into word processors.

The proposed system achieves a strong balance between computational efficiency and recognition accuracy. Quantitative evaluations confirm its capacity to preserve expression structure, while the Streamlit-based UI provides a practical interface for end-users. With further enhancements in token alignment and decoding logic, this framework can serve as a reliable foundation for applications in digital publishing, scientific content creation, and assistive technologies for mathematics education.

DISCUSSION

This project aimed to develop a computationally efficient and structurally accurate system for recognizing and converting LaTeX of printed and digital mathematical equations. The proposed architecture—built upon a lightweight convolutional encoder (LWDSC-SA), a BiLSTM sequence model, and a Luong attention-based decoder—was carefully designed to balance model complexity, training feasibility, and deployment practicality, particularly for mid-tier GPU environments like the RTX 2060.

The decision to base the encoder on depthwise separable convolutions, further enhanced by spatial attention mechanisms, was motivated by the need to reduce parameter count without compromising spatial feature richness. This proved effective, allowing the model to generalize well on equation structures with minimal training overhead. The BiLSTM was a lightweight yet capable sequence model, capturing left-to-right context dependencies within the encoded feature maps. While transformer-based alternatives have gained traction, the choice of BiLSTM was deliberate, given the model's interpretability, ease of training, and reduced memory footprint—an important consideration given the local hardware constraint.

Training the system on the Im2LaTeX-100k dataset required careful data preprocessing to handle the variability in image sizes and equation formatting. Image normalization, cropping, and resizing were crucial in ensuring input consistency, while tokenization and vocabulary mapping played a vital role in preserving syntactic integrity during LaTeX sequence generation. Despite the inherent challenges in aligning predicted sequences with ground truth, especially given LaTeX's sensitivity to structural tokens, the model achieved encouraging performance on several key metrics, including BLEU score, Character Recognition Rate (CRR), and normalized edit distance.

A notable aspect of the project was the comprehensive evaluation framework. Instead of relying solely on token-level accuracy, which tends to penalize even minor mismatches harshly, the analysis incorporated metrics that better reflect structural correctness and semantic coherence. This holistic evaluation approach provided a more realistic picture of the model's capabilities and limitations. While specific errors, such as incorrect operators or misplaced brackets, highlighted room for improvement, the overall system could capture the intended mathematical meaning in most cases, as demonstrated by high BLEU scores and qualitative visual fidelity.

Another key contribution of the project was the development of a fully functional Streamlit-based user interface. This component was not an afterthought but an integral part of the system's design, ensuring that the core model could be accessed, tested, and deployed in a user-centric manner. The UI allowed users to upload equation images and receive both LaTeX and MathML outputs, rendered visually for easy verification. The interface provided flexibility and transparency in inference behavior by supporting greedy and beam decoding modes. Furthermore, session caching, structured layouts, and real-time rendering enhanced usability and performance, making the system a research prototype and a deployable tool.

Despite the promising outcomes, the project encountered several limitations. Most notably, the model struggled with fine-grained token alignment, leading to low token accuracy scores despite producing visually correct outputs. This misalignment reflects a common challenge in sequence prediction tasks involving structurally dense output spaces like LaTeX. Additionally, while the model generalized well across most test samples, it occasionally failed on equations with deeply nested structures, rare symbols, or inconsistent formatting. These issues indicate a

need for broader vocabulary coverage and possibly more expressive sequence models in future iterations.

Looking ahead, there is ample scope for improvement and extension. Incorporating transformer-based modules or vision-language pretraining could help capture more global context and reduce alignment errors. Exploring alignment-aware or sequence-level loss functions could further mitigate strict token matching issues. On the deployment side, introducing UI feedback or correction support could enable continuous learning and domain adaptation. Finally, expanding the system's capabilities to handle handwritten equations or integrating it into larger document processing pipelines would make it even more versatile and impactful.

This project successfully demonstrates a complete pipeline for printed and digital equation recognition—from data preprocessing and model training to evaluation and real-time deployment. It highlights the potential of lightweight deep learning architectures in solving domain-specific OCR challenges. It provides a foundation for future research and development in intelligent document processing, academic publishing, and educational tools.

CONCLUSION

This project designed and implemented a robust, efficient, and end-to-end system for the recognition and LaTeX conversion of printed and digital mathematical equations. The proposed architecture, integrating a Lightweight Depthwise Separable Convolutional Encoder with Spatial Attention (LWDSC-SA), a BiLSTM-based sequence model, and a Luong Attention-driven decoder, demonstrated the feasibility of achieving structurally accurate LaTeX generation while maintaining computational efficiency. Trained and evaluated on the large-scale Im2LaTeX-100k dataset, the model showed promising results across multiple evaluation metrics, including BLEU score, Character Recognition Rate (CRR), and Normalized Edit Distance, with greedy decoding outperforming beam search in this specific setup.

Beyond the model, the project emphasized practical applicability by developing an interactive Streamlit-based user interface. This interface enabled seamless input-output interaction and real-time visualization of predictions, and it supported dual decoding strategies, thereby bridging the gap between model inference and real-world usability.

The system's strengths lie in its lightweight design, interpretability, and consistent structural accuracy, particularly for clean printed and digital equations. Although challenges remain in token-level alignment and handling complex nested structures, the pipeline offers a strong foundation for further advancement in mathematical OCR systems.

In conclusion, this work contributes a complete and deployable solution to the problem of automated equation recognition, balancing academic rigour with practical deployment considerations. It demonstrates that carefully crafted lightweight architectures can achieve high utility and performance in specialized domains with thoughtful training and user interface design. Future work can build on this foundation by incorporating more expressive models, alignment-aware objectives, and support for handwritten input, extending the system's capabilities and real-world impact.

REFERENCES

1. Öksüz, Özcan, Güdükbay, Uğur & Çetin, A. Enis. (2008). A video-based text and equation editor for LaTeX. *Engineering Applications of Artificial Intelligence*, 21(6), 952–960.
<https://doi.org/10.1016/j.engappai.2007.08.003>
2. Chang, Joseph, Gupta, Shrey, & Zhang, A. (2016). Painfree LaTeX with Optical Character Recognition and Machine Learning.
<https://api.semanticscholar.org/CorpusID:44007335>
3. Schechter, Amit, Borus, Norah & Bakst, William. (2017). Converting Handwritten Mathematical Expressions into LaTeX.
<https://api.semanticscholar.org/CorpusID:44420181>
4. Genthial, Guillaume & Sauvestre, Romain. (2017). Image to LaTeX.
<https://api.semanticscholar.org/CorpusID:229712014>
5. More, Avinash. (2018). Image to LaTeX via Neural Networks. Master's Projects. 602.
<https://doi.org/10.31979/etd.b52e-g3e7>
6. Kavyashree M K, Ranganath K N, Sirisha S. (2018). Development of Braille and LaTeX Markup Code for Mathematical Equations using OCR. *IJERT*, NCESC–2018, 6(13).
<https://doi.org/10.17577/IJERTCONV6IS13208>
7. Mishra, V. & Kaur, D. (2018). Sequence-to-Sequence Learning Using Deep Learning for Optical Character Recognition (OCR). *CSCI 2018*, pp. 324–329.
<https://doi.org/10.1109/CSCI46756.2018.00069>
8. Wang, Jian, Sun, Yunchuan & Wang, Shenling. (2019). Image To Latex with DenseNet Encoder and Joint Attention. *Procedia Computer Science*, 147, 374–380.
<https://doi.org/10.1016/j.procs.2019.01.246>
9. Wang, Hongyu & Shan, Guangcun. (2020). Recognizing Handwritten Mathematical Expressions as LaTeX Sequences Using a Multiscale Robust Neural Network.
<https://doi.org/10.48550/arXiv.2003.00817>
10. Wang, Zelun & Liu, Jyh-Charn. (2020). PDF2LaTeX: A Deep Learning System to Convert Mathematical Documents from PDF to LaTeX. *DocEng'20*, Article 4.
<https://doi.org/10.1145/3395027.3419580>
11. Singh, Sumeet & Karayev, Sergey. (2021). Full Page Handwriting Recognition via Image to Sequence Extraction.
<https://doi.org/10.48550/arXiv.2103.06450>
12. Garkal, A., Pal, A. & Singh, K. P. (2021). HMER-Image to LaTeX: A Variational Dropout Approach. *CICT 2021*, pp. 1–5.
<https://doi.org/10.1109/CICT53865.2020.9672359>

13. Wang, Z. & Liu, J.C. (2021). Translating math formula images to LaTeX sequences using deep neural networks with sequence-level training. *IJDAR*, 24, 63–75.
<https://doi.org/10.1007/s10032-020-00360-2>
14. Orji, Everistus Zeluwa, Haydar, Ali, Erşan, İbrahim & Mwambe, Othmar. (2023). Advancing OCR Accuracy in Image-to-LaTeX Conversion—A Critical and Creative Exploration. *Applied Sciences*, 13(22), 12503.
<https://doi.org/10.3390/app132212503>
15. Krishnan, Navaneetha M., Swetha, G., Saravanakumar, R., Nandhinishri, S.N. & Arthi, R. (2023). Comparative Analysis of Convolutional Neural Network and Character Recognition Techniques for Handwritten Mathematical Equation Solver. *Journal of Survey in Fisheries Sciences*, 10(4S), 1609–1632.
<https://sifisheressciences.com/journal/index.php/journal/article/view/1297/1328>
16. Sundararaj, Jayaprakash, Vyas, Akhil & Gonzalez-Maldonado, Benjamin. (2024). Automated LaTeX Code Generation from Handwritten Math Expressions Using Vision Transformer.
<https://doi.org/10.48550/arXiv.2412.03853>
17. Gurgurov, Daniil & Morshnev, Aleksey. (2024). Image-to-LaTeX Converter for Mathematical Formulas and Text.
<https://doi.org/10.48550/arXiv.2408.04015>
18. Sofi, A. Francis & Sangeetha, M. (2025). A comparison study on optical character recognition models in mathematical equations and in any language. *Results in Control and Optimization*, 18, 100532.
<https://doi.org/10.1016/j.rico.2025.100532>
19. Patil, Neeta P., Mane, Yogita D., Agrawal, Akshay, Vasoya, Anil, & Raut, Sanketi. (2025). Transformer-Based Image-to-LaTeX Conversion: Improving Mathematical Equation Recognition with Self-Attention. *JISEM*, 10(13s).
<https://doi.org/10.52783/jisem.v10i13s.2101>
20. Xia, R. et al. (2025). LaTeXNet: A Specialized Model for Converting Visual Tables and Equations to LaTeX Code. *ICASSP 2025*, pp. 1–5.
<https://doi.org/10.1109/ICASSP49660.2025.10887698>
21. Batool, A., Kim, J. & Byun, Y.C. (2025). A compact deep learning approach integrating depthwise convolutions and spatial attention for plant disease classification. *Plant Methods*, 21, 48.
<https://doi.org/10.1186/s13007-025-01325-4>

APPENDIX

```
In [18]: # ===== Load Best Model =====
ckpt = torch.load(checkpoint_path)
encoder.load_state_dict(ckpt["encoder"])
seq_model.load_state_dict(ckpt["seq_model"])
decoder.load_state_dict(ckpt["decoder"])
encoder.eval(); seq_model.eval(); decoder.eval()
print("Best model loaded for evaluation.")
```

Best model loaded for evaluation.

C:\Users\gurpr\AppData\Local\Temp\ipykernel_9032\2266255245.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
ckpt = torch.load(checkpoint_path)
```

```
In [19]: import re
import cv2
import numpy as np
from PIL import Image, ImageEnhance
import matplotlib.pyplot as plt
import torch
from latex2mathml.converter import convert as latex2mathml_convert

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
assert device.type == "cuda", "CUDA GPU is required for efficient training!"
```

```
In [20]: IMG_HEIGHT, IMG_WIDTH = 160, 1024

def preprocess_equation_image_for_inference(image_path):
    # 1. Load as grayscale
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Error loading image: {image_path}")

    # 2. Crop extra white spaces
    _, thresh = cv2.threshold(img, 240, 255, cv2.THRESH_BINARY)
    coords = cv2.findNonZero(255 - thresh)
    x, y, w, h = cv2.boundingRect(coords)
    cropped = img[y:y+h, x:x+w]

    # 3. Enhance sharpness & contrast
    pil_img = Image.fromarray(cropped)
    sharpness = ImageEnhance.Sharpness(pil_img).enhance(2.0)
    contrast = ImageEnhance.Contrast(sharpness).enhance(1.5)
    img = np.array(contrast)

    # 4. Binarization (optional)
    _, img = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    # 5. Resize to (160, 1024)
    img = cv2.resize(img, (IMG_WIDTH, IMG_HEIGHT), interpolation=cv2.INTER_AREA)

    # 6. Normalize and convert to tensor
    img_norm = img.astype(np.float32) / 255.0
    img_tensor = torch.from_numpy(img_norm).unsqueeze(0).unsqueeze(0) # [1, 1, 160, 1024]
    return img_tensor, img # Return both tensor and final processed image for display
```

```

In [21]: # ===== Example usage =====
image_path = r"C:\Users\gurpr\final_six_months_project\dataset\formula_images\4a8889f402.png"
#image_path = r"C:\Users\gurpr\Downloads\test_equations\3.png"
img_tensor, processed_img_for_plot = preprocess_equation_image_for_inference(image_path)

# ===== Model Inference =====
def predict_latex(image_tensor, encoder, seq_model, decoder, sos_token_id, eos_token_id, idx_to_token, max_len=160, device=device):
    encoder.eval(); seq_model.eval(); decoder.eval()
    with torch.no_grad():
        features = encoder(image_tensor)
        encoder_outputs = seq_model(features)
        batch_size = 1
        hidden = (
            torch.zeros(1, batch_size, decoder.dec_hidden_dim, device=device),
            torch.zeros(1, batch_size, decoder.dec_hidden_dim, device=device)
        )

    if mode == "greedy":
        input_token = torch.full((batch_size,), sos_token_id, dtype=torch.long, device=device)
        pred_ids = []
        for t in range(max_len):
            logits, hidden, _ = decoder(input_token, hidden, encoder_outputs)
            next_token = logits.argmax(dim=1)
            if next_token.item() == eos_token_id:
                break # Do not append EOS to output
            pred_ids.append(next_token.item())
            input_token = next_token
        pred_tokens = [idx_to_token.get(idx, "<UNK>") for idx in pred_ids]
        return " ".join(pred_tokens)

    elif mode == "beam":
        # Re-use your batch=1 logic (can call your beam_search_decode)
        decoded_tokens = beam_search_decode(
            encoder, seq_model, decoder, image_tensor.squeeze(0), # Remove batch dim if present
            sos_token_id, eos_token_id, beam_width=beam_width,
            max_len=max_len, device=device
        )
        pred_tokens = [idx_to_token.get(idx, "<UNK>") for idx in decoded_tokens]
        return " ".join(pred_tokens)
    else:
        raise ValueError("Unknown decoding mode. Use 'greedy' or 'beam'.")

```

```

In [22]: def predict_and_convert_mathml(
img_tensor, encoder, seq_model, decoder,
sos_token_id, eos_token_id, idx_to_token,
processed_img_for_plot=None,
max_len=160, beam_width=5, device="cuda",
save_path=None
):
    # Greedy Decoding
    greedy_pred = predict_latex(
img_tensor.to(device), encoder, seq_model, decoder,
sos_token_id, eos_token_id, idx_to_token,
mode="greedy"
)
    print("\nGreedy Decoding (LaTeX):", greedy_pred)
    try:
        greedy_mathml = latex2mathml_convert(greedy_pred)
        print("Greedy Decoding (MathML):")
        print(greedy_mathml)
        if save_path:
            with open(save_path.replace('.xml', '_greedy.xml'), 'w', encoding='utf-8') as f:
                f.write(greedy_mathml)
    except Exception as e:
        print("MathML conversion failed for Greedy:", e)

    # Beam Search Decoding
    beam_pred = predict_latex(
img_tensor.to(device), encoder, seq_model, decoder,
sos_token_id, eos_token_id, idx_to_token,
beam_width=beam_width, mode="beam"
)
    print("\nBeam Search Decoding (LaTeX):", beam_pred)
    try:
        beam_mathml = latex2mathml_convert(beam_pred)
        print("Beam Search Decoding (MathML):")
        print(beam_mathml)
        if save_path:
            with open(save_path.replace('.xml', '_beam.xml'), 'w', encoding='utf-8') as f:
                f.write(beam_mathml)
    except Exception as e:
        print("MathML conversion failed for Beam:", e)

    # Show the processed input image
    if processed_img_for_plot is not None:
        plt.figure(figsize=(10, 3))
        plt.imshow(processed_img_for_plot, cmap='gray')
        plt.title("Preprocessed Input for Model")
        plt.axis('off')
        plt.show()

    # Return all outputs for further processing
    return {
        "greedy_latex": greedy_pred,
        "greedy_mathml": greedy_mathml if 'greedy_mathml' in locals() else None,
        "beam_latex": beam_pred,
        "beam_mathml": beam_mathml if 'beam_mathml' in locals() else None,
    }

```


PROFORMA 4

Undertaking from the PG student while submitting his/her final dissertation to his respective institute

Ref. No. _____

I, the following student

Sr. No.	Sequence of students' names on a dissertation	Students name	Name of the Institute & Place	Email & Mobile
1.	First Author	Gurpreet Singh	SIG	Email: 23070243020@sig.ac.in Mobile: +91 9561224881

Note: Put additional rows in case of more number of students

Hereby give an undertaking that the dissertation **Automated Printed Equation Recognition and LaTeX/MathML Conversion from Images** has been checked for its Similarity Index/Plagiarism through the Turnitin software tool and that the document has been prepared by me, is my original work, and is free of any plagiarism. It was found that:

1.	The Similarity Index (SI) was: (Note: SI range: 0 to 10%; if SI is >10%, then authors cannot communicate ms; attachment of SI report is mandatory)	4%
2.	The ethical clearance for research work conducted was obtained from: (Note: Name the consent obtaining body; if 'not applicable', then write so)	NA
3.	The source of funding for research was: (Note: Name the funding agency, or write 'self' if no funding source is involved)	Self
4.	Conflict of interest: (Note: Tick ✓ whichever is applicable)	No
5.	The material (adopted text, tables, figures, graphs, etc.), as has been obtained from other sources, has been duly acknowledged in the manuscript: (Note: Tick ✓ whichever is applicable)	Yes

In case if any of the above-furnished information is found false at any point in time, then the University authorities can take action as deemed fit against all of us.

Full Name &
Signature of the student

Name &
Signature of SIU Guide/Mentor

Date: 11th July 2025

Place: Pune

Endorsement by
Academic Integrity Committee (AIC)

Note: It is mandatory that the Similarity Index report of plagiarism (only first page) should be appended to the UG/PG dissertation.

Turnitin Originality Report

Processed on: 16-Jun-2025 16:25 IST

ID: 2700352171

Word Count: 6590

Submitted: 1

v1.pdf By Gurpreet Singh

1% match
(Internet from
07-Apr-2025)

Similarity Index	Similarity by Source
4%	Internet Sources: 3% Publications: 4% Student Papers: 1%

<https://plantmethods.biomedcentral.com/articles/10.1186/s13007-025-01325-4>

< 1% match (Pradeep Singh, Balasubramanian Raman. "Deep Learning Through the Prism of Tensors", Springer Science and Business Media LLC, 2024)

[Pradeep Singh, Balasubramanian Raman. "Deep Learning Through the Prism of Tensors", Springer Science and Business Media LLC, 2024](#)

< 1% match (Internet from 01-Aug-2023)

<https://libraries.io/pypi/torchinfo/1.6.5>

< 1% match (Hakim Bouchal, Ahror Belaid, Farid Meziane. "Towards Accurate Recognition of Historical Arabic Manuscripts: A Novel Dataset and a Generalizable Pipeline", ACM Transactions on Asian and Low-Resource Language Information Processing, 2025)

[Hakim Bouchal, Ahror Belaid, Farid Meziane. "Towards Accurate Recognition of Historical Arabic Manuscripts: A Novel Dataset and a Generalizable Pipeline", ACM Transactions on Asian and Low-Resource Language Information Processing, 2025](#)

< 1% match (Internet from 07-Jan-2024)

<https://dokumen.pub/handbook-of-document-image-processing-and-recognition-1nbsped-9780857298584-9780857298591-9780857298607.html>

< 1% match (Internet from 17-Dec-2024)

<https://mbn.media/2020/11/48917/>

< 1% match (Internet from 04-Jun-2023)

https://erl-j.github.io/neural-music-instrument-cloning-web-supplement/Neural_Instrument_Cloning_PREPRINT.pdf

< 1% match (student papers from 25-May-2025)

[Submitted to Monash University on 2025-05-25](#)

< 1% match (student papers from 07-Mar-2021)

[Submitted to Aristotle University of Thessaloniki on 2021-03-07](#)

< 1% match (Kechi Zhang, Jia Li, Zhuo Li, Zhi Jin, Ge Li. "Transformer-based code model with compressed hierarchy representation", Empirical